BACHELOROPPGAVE:

# Snort Rule Manager

FORFATTERE:
Thomas Nyheim
Eirik Skogstad
Eigil Obrestad

DATO:
25.05.2014

# Sammendrag av Bacheloroppgaven

| | | |
|---|---|---|
| Tittel: | Snort Rule Manager | Nr: - |
| | | Dato: 25.05.2014 |
| Deltakere: | Thomas Nyheim | |
| | Eirik Skogstad | |
| | Eigil Obrestad | |
| Veiledere: | Slobodan Petrovic | |
| Oppdragsgiver: | Høgskolen i Gjøvik | |
| Kontaktperson: | Erik Helmås, erik.helmas@hig.no, 61135000 | |
| Stikkord | Norway, Norsk | |
| Antall sider: 12 | Antall vedlegg: | Tilgjengelighet: Åpen |

Kort beskrivelse av bacheloroppgaven:

This should be in Norwegian, I thought I would add some more text to make sure that it ran over multiple lines. I will need to check the page number feild as it probably should be just the core pages without the appendices. Currently it returns the last page of the whole document. If I cannot work it out, I will provide two inputs, gmtnumberpages and gmtappnumber. That should do the job.

# Summary of Graduate Project

| | | |
|---|---|---|
| Title: | Snort Rule Manager | Nr: - |
| | | Date: 25.05.2014 |
| Participants: | Thomas Nyheim | |
| | Eirik Skogstad | |
| | Eigil Obrestad | |
| Supervisor: | Slobodan Petrovic | |
| Employer: | Høgskolen i Gjøvik | |
| Contact person: | Erik Helmås, erik.helmas@hig.no, 61135000 | |
| Keywords | Thesis, Latex, Template, IMT | |
| Pages: 12 | Appendixes: | Availability: Open |
| Short description of the main project:<br>This is the short description of a bachelor thesis | | |

# Preface

We would like to thank Erik Hjelmås for motivating the development of a template for GUC's master's theses. This has expanded to include Bachelor and perhaps soon the PhD template.

# Contents

**Preface** . . . . . **iii**
**Contents** . . . . . **iv**
**List of Figures** . . . . . **v**
**List of Tables** . . . . . **vi**
**1 Introduction** . . . . . **1**
**2 Packages** . . . . . **2**
2.1 Packages Used by gucthesis . . . . . 2
2.2 Other Relevant Packages . . . . . 2
**3 Structural Elements** . . . . . **3**
3.1 Page Layout . . . . . 3
3.2 Fonts . . . . . 3
3.3 Sectioning Commands . . . . . 3
3.3.1 The subsection . . . . . 3
3.4 Floats (Figures and Tables) . . . . . 3
3.5 Quotes . . . . . 4
3.6 Lists . . . . . 4
3.7 Bibliographic References . . . . . 5
3.8 High-level use cases . . . . . 5
3.9 Detailed use cases . . . . . 8

# List of Figures

1 An example figure. . . . . 4

# List of Tables

1 An example table. . . . . 4

# 1 Introduction

Starting in 2005, Gjøvik University College(GUC) was given the right to issue Master diplomas. As a consequence of this, directions for the master's thesis have been developed [**?**], including guidelines for the typographical details. These detailed typographical rules have been implemented in the `gucmasterthesis` LaTeX document class.

The purpose of this document is to provide an example and description on how this class file can be used.

This was then extended to include bachelor project by Simon McCallum. The package has changed names and version number it is now called `gucthesis` v1.01 as of 2011/05/10.

# 2 Packages

The `gucthesis` is built upon the standard LaTeX `report` class. All commands from the `report` class can be used, with the two exceptions of `\subsubsection` and `\paragraph`. This is because there should only be three levels of headings according to the guidelines [**?**].

## 2.1 Packages Used by gucthesis

In addition to the `report` document class, `gucthesis` makes direct use of the following packages that must hence be present:

**geometry:** used for setting the sizes of the margins and headers.

**fontenc:** used with option `T1` for forcing the Cork font encoding (necessary for the Charter font).

**charter:** load Charter as the default font.

**euler:** load the Euler math fonts.

**babel:** to load language specific strings. Reasonable options include `british`, `american`, `norsk`, `nynorsk` and `samin`.

## 2.2 Other Relevant Packages

The author of a thesis might want to use a bunch of different packages to those described in Section 2.1 in order to have all features needed for their document. In particular, it is advised to use the following:

**inputenc:** to allow LaTeX to use more than 7-bit ASCII for its input. Most often, the option `latin1` will do.

**graphicx:** to include graphics.

**hyperref:** this is a very nice package that makes cross links in pdf documents. Use with option `dvips` or `pdftex` in accordance with the driver that you use. Unfortunately, hyperref is not completely bugfree...

# 3 Structural Elements

The title of the thesis should be set using the `\thesistitle` command, and the date of the thesis should be set using the `\thesisdate` command. This makes the title and date appear in the running header, like in this document.

## 3.1 Page Layout

The geometry of the page has been set using the `\geometry` command.

## 3.2 Fonts

Due to limited LaTeX support for the Georgia font, Charter has been chosen instead. For mathematical formula, the Euler fonts are used, since they blend more nicely with the Charter than the standard LaTeX fonts:

$$f(x) = \int_0^x g(\tau)\,d\tau$$

For inline math you can use `\(` and `\)` for example $f(x) = \frac{x^2}{1+x^2}$. This also allows you to use `/` and `\`. You need to include the `{}` when you want the special character to have other letters immediately after it.

## 3.3 Sectioning Commands

The standard LaTeX sectioning commands are used for both numbered and unnumbered sections. The top level is given by the `\chapter` command. This starts a new right page. The two lower levels are obtained using the `\section` and `\subsection` commands. The standard LaTeX `\subsubsection` and `\paragraph` commands have been disabled since their use is not encouraged by the thesis guidelines. When you use these they will not be given numbers. They still appear in the document with highlighting but not in the table of contents.

### 3.3.1 The subsection

This is an example of a subsection.

#### The subsubsection

This is an example of a subsubsection.

*The paragraph*

This is an example of a paragraph with a heading.

## 3.4 Floats (Figures and Tables)

Figures are placed in the `figure` environment. An example is shown in Figure 1. Tables are placed in the `table` environment. An example is given in Table 1. Figures and tables float freely around in the document in accordance with standard LaTeX behavior.

The captions are placed *below* both for the figures and the tables. The caption is set in 9pt. If the caption is shorter than one line, it is centered.



Figure 1: An example figure. If the caption is shorter than one line, it is centered. If it goes over more than one line, it is left and right justified. Furthermore, it is suggested that an alternative short caption is given in order to produce a good list of figures.

| Age | IQ |
|---|---|
| 10 | 100 |
| 20 | 100 |
| 30 | 150 |
| 40 | 100 |
| 50 | 100 |

Table 1: An example table.

## 3.5 Quotes

Quotes are inserted using the standard LaTeX `quote` environment. The environment has been changed so that a 9pt font is used:

> "And I looked, and, behold, a whirlwind came out of the north, a great cloud, and a fire infolding itself, and a brightness was about it, and out of the midst thereof as the colour of amber, out of the midst of the fire. Also out of the midst thereof came the likeness of four living creatures."

## 3.6 Lists

Point lists and enumerated lists are made by using the standard `itemize` and `enumerate` environments, respectively. The spacing is going to be changed in accordance with the specification. For `itemize`, the results look like this:

- First item.
- Second item. Here I will put some long text, just to illustrate. Here I will put some long text, just to illustrate. Here I will put some long text, just to illustrate. Here I will put some long text, just to illustrate.
- Third item also has subitems:
  - First subitem.
  - Second subitem.
  - Third subitem.

and for `enumerate` like this:

1. First item.
2. Second item. Here I will put some long text, just to illustrate. Here I will put some long text, just to illustrate. Here I will put some long text, just to illustrate. Here I will put some long text, just to illustrate.
3. Third item also has subitems:
   1. First subitem.
   2. Second subitem.
   3. Third subitem.

   You may also want to use descriptive lists

**First** the first item.

**Second** the second item. Here I will put some long text, just to illustrate. Here I will put some long text, just to illustrate. Here I will put some long text, just to illustrate. Here I will put some long text, just to illustrate.

**What now** the third item also has subitems:

1. First subitem.
2. Second subitem.
3. Third subitem.

## 3.7 Bibliographic References

You should cite articles [**?**], books [**?**], anthologies [**?**] and web publications [**?**] like this. There is always an issue referencing web pages. Currently we suggest that you use the HiG Website [**?**].

A particular bibliography style file for GUC named `gucthesis.bst` has been developed based upon the standard BibTEX `unsrt` style.

article usecases fullpage

## 3.8 High-level use cases

| **Use case** | **Manual rule update** |
|---|---|
| *Primary actor* | User |
| *Purpose* | Update the central rule database |
| *Description* | The user sends a command to trigger a rule update, and the system pulls rule files from all the sources specified in the list of sources. Rules are then processed and stored in the database. |
| *Preconditions* | One or more sources must exist |
| *Postconditions* | The central rule database is up to date with the newest rules from all sources |

| **Use case** | **Automatic rule update** |
|---|---|
| *Primary actor* | System |
| *Purpose* | Update the central rule database |
| *Description* | At configured time-intervals, the system is triggered to update and pulls rule files from all the sources specified in the list of sources, processes the rules and then stores them in the database. |
| *Preconditions* | One or more sources must exist |
| *Postconditions* | The central rule database is up to date with the newest rules from all sources |

| **Use case** | **Treshold is applied/removed to rule** |
|---|---|
| *Description* | User sends a command with either tresholding parameters or removal of threshold for a rule and the system updates the database to reflect the change. One threshold parameter per rule. |

| **Use case** | **Suppression is applied to rule** |
|---|---|
| *Description* | User sends a command with either suppression parameters or removal of threshold for a rule and the system updates the database to reflect the change. Suppression parameters can take multiple IP-addresses/IP-ranges. |

| **Use case** | **Viewing rules** |
|---|---|
| *Description* | Rules are retrieved from the database, processed for GUI/CLI format and then delivered to GUI or CLI in specified format. |

| **Use case** | **User comments on rule or rule set** |
|---|---|
| *Description* | User specifies a rule or rule set and applies a comment to it, that is then stored in the database. |

| **Use case** | **User edits/deletes a comment on a rule or rule set** |
|---|---|

| | |
|---|---|
| *Description* | User specifies a rule or rule set and edits or deletes it from the database. |

| **Use case** | **User registers a sensor** |
|---|---|
| *Description* | User sends parameters for a sensor and the system stores the parameters in the database. |

| **Use case** | **User registers a source for rule updates** |
|---|---|
| *Description* | User sets parameters for sources and the system stores the parameters in the database. |

| **Use case** | **System verifies rules as valid and functional** |
|---|---|
| *Description* | Before rules are stored in the database, the system process and check them for validity and functionality by doing a test run in a local SNORT to catch any errors they might produce and register in the database |

| **Use case** | **User organizes sensors into sensor group** |
|---|---|
| *Description* | A user sets up a sensor group and then includes one or more sensors into this group. The user can then manipulate rules on a sensor group basis. |

| **Use case** | **Rules are distributed by the user** |
|---|---|
| *Description* | User sends a command to distribute to specified sensors and the system retrieves rule information for the database, processes them and formats them for SNORT, then distributes them to all sensors specified per their individual settings. |

| **Use case** | **Rules are distributed by the system** |
|---|---|

| | |
|---|---|
| *Description* | Configured at intervals, the system triggers a command to distribute to specified sensors and the system retrieves rule information for the database, processes them and formats them for SNORT, then distributes them to all sensors specified per their individual settings. |

## 3.9 Detailed use cases

| **Use case** | **Central rule database update** |
|---|---|
| *Primary actor* | System |
| *Purpose* | Update the central rule database from external sources |
| *Description* | 1. The case is triggered when either the user issues the command to update, or the system (daemon) issues the command based on a set time-interval. The user has the option to update rules from a local file, if this is the case, the program will proceed from part 3.<br>2. The process starts in the Rule Downloading Module where each rule source is checked for updates by comparing MD5 values. If the MD5 values differ, the module will retrieve the updated rule files and pass them along to the Rule Pre-Storage Processing Module.<br>3. The Rule Pre-Storage Processing Module then begins reading the files line by line and parses metadata from the new rule files. The rules are checked against the current rules stored in the database to identify which rules are new or modified. Based on this identification, the new or modified rules are parsed into complete objects and passed to the Database/Core Module that stores the new or modified rules into the database. One or more rule sources must be registered in the system. Alternatively, one or more local rule files must be present. Rules have either been confirmed as up-to-date or the new or updated rule objects have been passed to the Database Module. |
| *Preconditions* | |
| *Postconditions* | |
| *Triggers* | Manual rule update, Automatic rule update |
| *Basic Flow:* | |

1. Update is triggered
2. Rule Downloading Module compares source with stored rules.
3. Updated rule files are downloaded from sources. The files are validated and passed to the Pre-Storage Processing Module module.
4. The new files are parsed for rule SID and revision number.
5. A list of rule SID and revision numbers are gathered from the database and these are compared to identify which rules are new or modified.
6. New or changed rules are then parsed completely into full objects and these objects are passed to the Database/Core Module.
7. The Database/Core Module stores the new or changed rule objects into the database.

---

*Extensions:*

1. 1. Update is triggered by user
   2. Update is triggered by system
3. 1. The rules at source is equal to the stored rules. Database is up to date, so no more action is required.
   2. The rules at source is newer than stored rules. Database must be updated.
3. 1. User is notified that database is up to date.
   2. User is notified that the database is not up to date and an update process has begun.
3. 1. The user is notified that the update is complete.
   2. The user is notified that rules have been updated during since the last scheduled update interval.

---

*Error handling:*

1. The source and database checksum is different but the source rules are older than what is stored in the database: *The program can still continue, because if the SIDs already exist they will not be added again and if the revisions are older the rules will also not be updated.*
2. A ruleset has changed its name or one or more rules are moved to a different ruleset: *Based on a configuration file, the affected rules will either be a) automatically moved to the new ruleset or b) stay put in their original ruleset. The user will be notified and can choose to reverse or distribute the changes.*

---

| **Use case** | **Globally enable/disable a rule or ruleset** |
|---|---|
| *Primary actor* | User |
| *Purpose* | Turn a rule or ruleset on/off |

| | |
|---|---|
| *Description* | User sends a command to enable/disable a specified rule or ruleset, and the system updates the database to reflect that the rule is enable/disabled. |
| *Preconditions* | The rule or ruleset must exist in the database |
| *Postconditions* | The rule or ruleset is deativated on all sensors **except** if this setting is overridden in a sensor's local RuleModifier. |

| **Use case** | **Locally enable/disable a rule or ruleset on a sensor** |
|---|---|
| *Primary actor* | User |
| *Purpose* | Turn a rule or ruleset on/off on a sensor |
| *Description* | A user wants to either enable or disable a rule or rule set on one or more sensors. This is done by sending a command that specifies which rule or rule set is to be affected, how it is affected and on which sensor or sensors are affected. The central database must be updated with this new status and the change must be pushed out to the affected sensor or sensors. |
| *Preconditions* | One or more rule sets or rules must be present in the central database. One or more sensor must be registered. |
| *Postconditions* | The changes to the rule or rule set must be reflected on the affected sensor or sensors. |
| *Triggers* | |

*Basic flow*

1. The user enables/disables a rule on a given sensor.
2. The central database changes the status of the rule or rule set by manipulating the status flags.
3. The system notifies the sensor that a change has occurred.
4. The change is synced to the affected sensor.

*Error handling:*

1. The change may never be synced to the sensor due to errors: *There must be an error protocol that is followed in this scenario.*

| **Use case** | **Rule is modified/de-modified with a threshold or suppression on one or more sensors** |
|---|---|
| *Primary actor* | User, System |

| | |
|---|---|
| *Purpose* | Manipulate configurations for specific rules on specific sensors. |
| *Description* | A user wants to either turn on or off a threshold or suppression modifier on a rule on one or more sensors. This is done by sending a command that specifies which rule the modification will apply to, the parameters of the modification and for which sensor or sensors this will apply for. The central database must be updated with this new status and the change must be pushed out to the affected sensor or sensors. |
| *Preconditions* | One or more rule sets or rules must be present in the central database. One or more sensors must be registered. If a modification is applied, there must not be another modification for the same rule on the same sensor. If a modification is removed, it must already exist. |
| *Postconditions* | The modifications must be applied on the rule and sensor or sensors in question. |
| *Triggers* | |

*Basic flow*

1. User sets a threshold/suppression on a rule on a sensor.
2. The central database creates a threshold/suppression for the specified rule and the specified sensor.
3. The system notifies the sensor that a change has occurred.
4. The change is synchronized to the affected sensor.

*Extensions*

1. User turns off a threshold/suppression on a rule on a sensor
2. The central database deletes the threshold/suppression for the specified rule and the specified sensor.

*Error handling*

1. The change may never be synced to the sensor due to errors: *There must be an error protocol that is followed in this scenario.*

| **Use case** | **Rules are distributed** |
|---|---|
| *Primary actor* | User, System, Sensor |
| *Purpose* | Distribute rules from the central database to the local sensors |

| | |
|---|---|
| *Description* | A user or the system wants to synchronize the rules stored in the central database to either one, multiple or all sensors. This is initiated either by the user sending a command or by the system having a regular interval. The system must then calculate and organize the central database data so that the affected sensors get synchronized. |
| *Preconditions* | One or more rule sets or rules must be present in the central database. One or more sensors must be registered |
| *Postconditions* | The rules on a local sensor must be in sync with the central database. |
| *Triggers* | |

*Basic flow*

1. Synchronize signal is sent.
2. The system calculates and organizes the data for each sensor affected by the synchronization.
3. The system initializes a synchronization protocol with the affected sensor or sensors, sending a list of what the sensor should have.
4. The sensor calculates diff between should have and has.
5. Compare diff.
6. Send update.
7. The sensor includes the new changes into its database and files. Tells the central system that it is now up to date.
8. The system carries out maintenance to reflect the synchronization.
9. The user is notified that all sensors affected are now up to date.

*Extensions*

1. 1. User sends a command to synchronize one, more or all sensors

   2. The system reaches an interval for synchronizing all sensors.
   enumi4
3. 1. The sensor content is different from what it should have, asks central system to send what it needs.

   2. The sensor content is not different from what it should have, no further action is required. This status is sent to the central system.
3. 1. The system retrieves and sends what the sensor asks for.

   2. The system terminates the synchronization protocol with the sensor in question.

*Error handling*

1. The change may never be synced to the sensor due to errors. Solution: There must be an error protocol that is followed in this scenario.